

```

1  (*
2      CS51 Lab 6
3      Variants, algebraic types, and pattern matching (continued)
4
5  In this lab, you will continue to use algebraic data types to create
6  several data structures. You'll create a data structure for storing
7  weather information and work with a specific type of binary tree, the
8  *binary search tree*, which allows for efficient storage and search of
9  ordered information. A particular application is the use of Gorn
10 addresses, named after the early computer pioneer Saul Gorn of
11 University of Pennsylvania, who invented the technique.
12
13 There are two parts to this lab. Please refer to the following
14 files to complete all exercises:
15
16 -> lab6_part1.ml -- Part 1: The Weather
17     lab6_part2.ml -- Part 2: Binary search trees and Gorn addresses
18 *)
19
20 (*=====
21 Readings:
22
23     This lab builds on material from Chapter 11.3 to the end of Chapter
24     11 of the textbook <http://book.cs51.io>, which should be read
25     before the lab session.
26
27 Objective: This lab is intended to reinforce core concepts in
28 typing in OCaml, including:
29
30     Algebraic data types
31     Using algebraic data types to enforce invariants
32     Implementing polymorphic algebraic data types
33 =====*)
34
35 (*=====
36     Part 1: The Weather
37     Variants and invariants revisited
38
39 As the saying goes, everybody talks about the weather, but nobody
40 *does* anything about it. Let's remedy that. We'll implement a data
41 structure for storing seasonal weather information.
42
43 .....
44 Exercise 1: Define a season type
45
46 There are four seasons -- spring, summer, autumn, and winter. Define an
47 enumerated type that allows for just these four values.
48 .....*)
49
50 type season = ToBeDefined
51
52 (*.....

```

```

53 Exercise 2: Define a type providing the weather condition
54
55 For our purposes, there are three weather conditions -- sunny, rainy,
56 and snowy. Each condition (except for "sunny", of course) has an
57 associated precipitation amount, an integer specifying the
58 precipitation in millimeters.
59
60 Define a type providing the possible weather conditions. Remember the
61 invariant: no precipitation amount for sunny conditions.
62 .....*)
63
64 type condition = ToBeDefined
65
66 (*.....
67 Exercise 3: Define a weather status type combining season and condition
68
69 The `weather_status` type should combine a season together with a
70 condition. Use a record type for the combination, with appropriate
71 field names.
72 .....*)
73
74 type weather_status = ToBeDefined
75
76 (* For consistency with our unit testing, we should make sure that the
77 types you defined match with the types we're testing against. You
78 should call over a staff member to verify the types you've
79 specified. Alternatively, we've placed our expected data types at
80 url.cs51.io/lab6types *)
81
82 (*.....
83 Exercise 4: For each of the following cases, construct a value of type
84 `weather_status` that represents the case. If no such value exists,
85 specify a value representing a sunny day in summer.
86
87 Case 1: a sunny day in winter with 2 mm of snow
88 Case 2: a fall day with 5 mm of rain
89 Case 3: a snowy winter day with 100 mm of snow
90 .....*)
91
92 let case1 = failwith "case 1 not implemented" ;;
93 let case2 = failwith "case 2 not implemented" ;;
94 let case3 = failwith "case 3 not implemented" ;;
95
96 (* Now for some functions that manipulate these weather values.
97
98 .....
99 Exercise 5: Calculating precipitation
100
101 Define a function `precipitation_amount : condition -> int` that given
102 a weather condition, returns the precipitation amount (0 mm for sunny
103 days of course).
104 .....*)
105
106 let precipitation_amount (condition : condition) : int =

```

```

107     failwith "precipitation_amount not implemented"
108
109 (*.....
110 Exercise 6: Strings for a season
111
112 Define a function `season_to_string : season -> string` that given a
113 season, returns a string naming that season. This may come in handy in
114 Exercise 7.
115 .....*)
116
117 let season_to_string (season : season) : string =
118     failwith "season_to_string"
119
120 (*.....
121 Exercise 7: Describing the weather
122
123 Define a function `describe_weather : weather_status -> string` that
124 given a weather status, returns a string describing the weather. Here
125 are some example strings that the function should return given
126 appropriate inputs.
127
128     "It's snowing in winter. Precipitation: 20 mm."
129     "It's a sunny summer day."
130     "It's raining in autumn. Precipitation: 100 mm."
131 .....*)
132
133 let describe_weather (status : weather_status) : string =
134     failwith "describe_weather not implemented"

```